

Nodejs session 5

▼ Coding

```
// Index.js
const express = require('express');
const app = express();
const { title } = require('node:process');
const port = 3000;

app.use(express.json()); // Middleware to parse JSON request bodies

const coursesRouter = require('./routes/courses_routes');
app.use('/api/courses', coursesRouter);

app.listen(port, () => {
  console.log(`Server is running on http://localhost:${port}`);
});

//courses_routes.js

const express = require('express');

const {body} = require('express-validator');

const router = express.Router();

const {
  getAllCourse,
  getCourseById,
  postCreateCourse,
  updateCourseById,
  deleteCourseById
```

```

} = require('../conrollers/courses-conroler');
// Get all courses
router.route('/')
  .get(getAllCourse)
  .post([
    body('title')
      .notEmpty()
      .isLength({ min: 2})
      .withMessage('Title must be at least 2 characters long'),
    body('price')
      .notEmpty()
      .withMessage('Price must be a positive number')
  ], postCreateCourse)

// Get a specific course by ID
router.route('/:id')
  .get(getCourseById)
  .patch(updateCourseById)
  .delete(deleteCourseById);

module.exports = router;

// courses-conroler.js
// Import courses data --> /Data/courses.js
let { courses } = require('../Data/courses');

const {validationResult} = require('express-validator');

const getAllCourse = (req, res) => {
  res.json(courses);
}

```

```

const getCourseById = (req, res) => {
  const course = courses.find(c => c.id === parseInt(req.params.id));
  if (!course) return res.status(404).json({ message: 'Course not found' });
  res.json(course);
}

const postCreateCourse = (req, res) => {
  const course = {id: courses.length + 1, ...req.body};
  courses.push(course);
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    return res.status(400).json({ errors: errors.array() });
  }

  res.status(201).json(course);
};

const updateCourseById = (req, res) => {
  let course = courses.find(c => c.id === parseInt(req.params.id));
  if (!course) return res.status(404).json({ message: 'Course not found' });
  course = { ...course, ...req.body };
  res.status(200).json(course);
};

const deleteCourseById = (req, res) => {
  const course = courses.find(c => c.id === parseInt(req.params.id));
  if (!course) return res.status(404).json({ message: 'Course not found' });
  courses = courses.filter(c => c.id !== parseInt(req.pa

```

```

    rams.id));
    res.json({ message: 'Course deleted' });
  });

module.exports = {
  getAllCourse,
  getCourseById,
  postCreateCourse,
  updateCourseById,
  deleteCourseById
}
// Data --> courses.js
let courses = [
  { id: 1, title: 'Js Course 1', price: 100 },
  { id: 2, title: 'React Course 2', price: 150 },
  { id: 3, title: 'Html Course 3', price: 200 }
];

module.exports = {
  courses
};

```

▼ Explanation

1. الاستيراد (Imports) ●



```
const express = require('express');
```

نظري 🧠

- بنستورد مكتبة **Express**
- دي framework بتسهل التعامل مع السيرفر

عملي 💻

بدون السطر ده:

```
const app = express();
```

هيديك error → express is not defined ❌

بيحصل إيه ⚙️

Express ويحمل مكتبة `node_modules` بيدور في Node



```
const app = express();
```

نظري 🧠

- بنعمل instance (نسخة) من Express

- ده هو السيرفر بتاعك `app`

عملي 💻

كل حاجة بعد كده:

```
app.get()  
app.post()
```

بتشتغل عليه



middleware و routing زي functions فيه object بيرجع Express



```
const {body, validationResult}=require('express-validator');
```

نظري 🧠

- بنستورد أدوات validation

- `body` → نتحقق من الداتا اللي جاية
- `validationResult` → ولا لأ errors نشوف فيه

عملي 💻

```
body('title').notEmpty()
```

→ يمنع إرسال title فاضي



بيعمل middleware بيشيك على request قبل ما يكمل



```
const { title }=require('node:process');
```

نظري 🧠

- بيستورد حاجة اسمها title من process (غالباً مش مستخدمة هنا ❌)



السطر ده مالوش لازمة في الكود ده

→ ممكن تشيله عادي



```
const port=3000;
```

نظري 🧠

- البورت اللي السيرفر هيشغل عليه

عملي 💻

تفتح:

http://localhost:3000

2. Middleware



```
app.use(express.json());
```

نظري 

Middleware JSON → Object بھوّل

عملي 

لو بعت request:

```
{
  "title": "JS",
  "price": 100
}
```

تقدر تستخدم:

```
req.body.title
```

لو شيلته 

```
req.body
```

3. Database (Fake)



```
let courses = [...]
```

نظري 🧠

- array بتمثل database fake

عملي 💻

```
courses.push(newCourse);
```

→ كأنك بتضيف في DB

4. GET All



```
app.get('/api/courses', (req, res) => {  
  res.json(courses);  
});
```

نظري 🧠

- بيرجع كل الكورسات Route

عملي 

تفتح:

```
GET /api/courses
```

→ بيرجع JSON



- `req` → request
- `res.json()` → JSON يحوّل ل

5. GET by ID



```
app.get('/api/courses/:id', (req,res) => {
```

نظري 

`:id` → dynamic param



```
const course = courses.find(c => c.id === parseInt(req.params.id));
```



• بندور على course بال id

عملي 

```
GET /api/courses/2
```

→ يجيب الكورس رقم 2



URL بيحيب الرقم من `req.params.id` →



```
if (!course) return res.status(404).json({ message: 'Course not found' });
```



لو مش موجود → error



لو دخلت id غلط → 404

6. POST (Create)



```
app.post('/api/courses', [ ... ], (req,res) => {
```



- إنشاء course جديد
- فيه validation middleware



(Validation)

```
body('title').notEmpty().isLength({ min:2 })
```



لو بعت:

```
{ "title":"" }
```

✗ error



```
const course = {id:courses.length+1, ...req.body};
```



- بي عمل object جديد

- `...req.body` = spread



```
{ "title":"JS", "price":100 }
```

→

```
{id:4,title:"JS",price:100 }
```



```
courses.push(course);
```



بيضيفه في array



```
const errors=validationResult(req);
```



بيشوف فيه errors ولا لا



```
if (!errors.isEmpty())
```



لو فيه error → يرجعه

⚠️ غلطة في الكود عندك 🔥

أنت بتعمل:

```
courses.push(course);
```

❌ validation قبل

→ المفروض validation الأول

7. PATCH (Update)



```
app.patch('/api/courses/:id', (req, res) => {
```



تعديل جزئي



```
course= { ...course, ...req.body };
```



لو بعت:

```
{ "price":300 }
```

→ يغير السعر بس

⚠️ مشكلة:

✗ مش بيحدث array فعلياً
(لازم ترجعه مكانه)

8. DELETE



```
app.delete('/api/courses/:id', (req, res) => {
```



مسح course



```
courses=courses.filter(c =>c.id!==parseInt(req.params.id));
```



بيمسح العنصر

9. تشغيل السيرفر




```
app.listen(port, () => {
```



يشغل السيرفر



```
Server is running on http://localhost:3000
```

أهم جزء طلبته (PARAMS) 🔥

يعني إيه Params ؟ 🧠

Params = URL بيانات بتيجي في

مثال 📖

```
/api/courses/5
```

→ هنا:

```
req.params.id=5
```

فائدتها 🧠

• تجيب resource معين

- تعمل filtering

- ديناميكي routing

🔥 الفرق بين params و body و query

النوع	مثال	استخدام
params	<code>/courses/1</code>	ID
query	<code>/courses?price=100</code>	فلتر
body	JSON	POST / PATCH

🐱‍💻 ك Pentester ⚠️

params تكون ممكن:

- IDOR
- Injection
- Missing validation

🎯 خلاصة مهمة

- routing بسيط Express
- Middleware = قلب النظام
- insert لازم قبل Validation
- params = dynamic routing